Ashokvatika Mobile App Release Audit

Role mix: senior full-stack engineer, performance auditor, QA lead, UX critic, and security reviewer.

Scope and method

This audit is based on direct inspection of the Android codebase in the workspace. Findings are limited to what could be verified from source. Where backend or runtime behavior could not be proven from the repository alone, the item is marked Unverified with a concrete verification step.

Critical Findings

1. Hardcoded admin credentials and bypassed auth store

Severity: Critical
Location: app/src/main/java/com/ashokvatika/ui/LandingLoginScreen.kt - LandingLoginScreen()

Diagnosis: The production login screen is seeded with username `admin` and password `123`, and login succeeds when those exact hardcoded values are entered. The stronger `AdminAuthStore` implementation exists in the repo but is not wired into the screen at all. This means anyone with the APK or a screenshot of the code path can access the admin surface.

Impact: Security, revenue risk, unauthorized inventory changes, unauthorized approval/denial of requests, maintainability.

Suggested amendment: Replace the hardcoded credential check with a real auth source. At minimum, wire `LandingLoginScreen` to `AdminAuthStore` or Firebase Auth and remove default seeded secrets from UI state.

Exact code change where possible:

```
// LandingLoginScreen.kt
val authStore = remember(context) { AdminAuthStore(context.applicationContext) }
var username by remember { mutableStateOf("") }
var password by remember { mutableStateOf("") }

fun submit() {
    val ok = username == expectedAdminUser && authStore.verifyPin(password)
    if (ok) onLoggedIn() else error = "Invalid credentials"
}
```

Test to confirm the fix: Launch the app on a fresh install. Verify that `admin / 123` no longer grants access unless explicitly provisioned. Add a UI test that enters invalid credentials and confirms the route does not advance.

2. Payment flow is a non-verified stub but still closes deals

Severity: Critical
Location: app/src/main/java/com/ashokvatika/ui/admin/AdminScreens.kt - PaymentGatewayDialog(), app/src/main/java/com/ashokvatika/ui/AppRoot.kt - onSaveDeal

Diagnosis: The payment dialog renders a static `dummy_qr` image and exposes a `Close deal and clear cart` action without any payment-status verification, receipt capture, or required customer-data validation. A user can finalize and archive a sale even if no payment happened.

Impact: Revenue risk, auditability, UX trust, operational reconciliation risk.

Suggested amendment: Gate deal closure behind an actual payment confirmation state. Require at least transaction reference + customer identity, and disable close until payment is verified. If payment integration is not ready, remove the close action from production builds.

Exact code change where possible:

```
data class PaymentState(
    val qrUrl: String,
    val verified: Boolean,
    val providerTxnId: String = ""
)

ActionButtonCard(
    title = "Close deal and clear cart",
    color = AccentGreen,
```

```
    onClick = onCloseDealAndClearCart,
    enabled = paymentState.verified && clientName.isNotBlank() && clientPhone.length >= 10,
)
```

Test to confirm the fix: Attempt to close a deal with blank client fields and without a verified payment callback. Expected result: action stays disabled and no deal is added to closed deals.

High Findings

1. Open cart and saved deals are volatile in-memory state

Severity: High
Location: app/src/main/java/com/ashokvatika/ui/AppRoot.kt - AppRoot()

Diagnosis: Open cart rows, saved deals, client details, and transaction code are stored only in `remember` / `mutableStateListOf` state. They are not persisted to disk, not in a ViewModel, and not even `rememberSaveable`. Rotation, process death, task removal, or low-memory reclaim can wipe active sales.

Impact: Revenue risk, UX, data loss, support burden.

Suggested amendment: Move cart/deal state into a ViewModel backed by Room/DataStore or Firestore draft documents. At minimum use `rememberSaveable` with parcelable DTOs for config changes, but production should persist drafts durably.

Exact code change where possible:

```
class CartViewModel(
    private val repository: CartDraftRepository
) : ViewModel() {
    val openCart = repository.observeDraftCart().stateIn(...)
    suspend fun saveDraft(rows: List<OpenCartRow>) = repository.saveDraft(rows)
}
```

Test to confirm the fix: Add items to the cart, rotate the device, then force-stop/relaunch after backgrounding. Expected after fix: cart and client fields are restored.

2. Pricing inputs are used in calculations but not persisted correctly

Severity: High
Location: app/src/main/java/com/ashokvatika/ui/admin/AdminScreens.kt - CategoryScreen(), buildInventoryItem(); app/src/main/java/com/ashokvatika/data/AdminFirestoreRepository.kt - upsertInventoryItem()

Diagnosis: Vendor name, phone, email, and `landingPriceText` are collected and used for price calculations, but `buildInventoryItem()` does not store them on `InventoryItem`. In Firestore, `landingPrice` is written from `item.checkout`, which is a different field. Reopening an item loses the true landing-price inputs and can produce incorrect margin calculations.

Impact: Revenue risk, pricing integrity, maintainability, QA regression risk.

Suggested amendment: Extend `InventoryItem` and Firestore serialization with explicit vendor and landing-price fields. Stop reusing `checkout` as `landingPrice`.

Exact code change where possible:

```
data class InventoryItem(
    ...,
    val vendorName: String = "",
    val vendorPhone: String = "",
    val vendorEmail: String = "",
    val landingPrice: String = "0",
)

// AdminFirestoreRepository.kt
"landingPrice" to item.landingPrice,
"vendorName" to item.vendorName,
"vendorPhone" to item.vendorPhone,
"vendorEmail" to item.vendorEmail,
```

Test to confirm the fix: Create a product with vendor details and landing price, save it, reopen the same item, and verify every field and derived margin value is restored exactly.

3. Firestore documents are carrying image payloads and can hit size/performance limits

Severity: High
Location: app/src/main/java/com/ashokvatika/data/AdminFirestoreRepository.kt - upsertInventoryItem(), encodeImageUriToBase64()

Diagnosis: Inventory writes embed `imageBase64` directly inside the Firestore document while also storing `mediaUris`. This duplicates media metadata into a document store that has a 1 MiB document limit and poor characteristics for binary payloads. Multi-image products and repeated writes will become slow, costly, and brittle.

Impact: Performance, scalability, storage cost, sync reliability.

Suggested amendment: Upload media to Firebase Storage (or equivalent) and keep only URLs, thumbnail URLs, width/height, and content hashes in Firestore. Remove `imageBase64` from product documents.

Exact code change where possible:

```
// Firestore document shape
"thumbnailUrl" to uploaded.thumbnailUrl,
"media" to uploaded.items.map { mapOf("url" to it.url, "type" to it.type) }
// Remove imageBase64 entirely
```

Test to confirm the fix: Upload several large images to one product and verify document writes stay fast, document size remains small, and Firestore writes do not fail with size-related errors.

4. Monolithic Compose screen file has already caused compiler OOM and will keep slowing delivery

Severity: High
Location: app/src/main/java/com/ashokvatika/ui/admin/AdminScreens.kt - whole file (7,782 lines), especially CategoryScreen(), OtcScreen(), OpenCartScreen(), MediaGalleryPopup()

Diagnosis: The admin UI is concentrated in one very large file with multiple state-heavy composables and business logic mixed into rendering. This has already manifested as Kotlin FIR / data-flow analyzer `OutOfMemoryError` during compilation. Even when it builds, this structure raises regression risk and slows every future change.

Impact: Build stability, developer velocity, maintainability, QA risk.

Suggested amendment: Split the file by feature and hoist business state into dedicated state holders / ViewModels. Keep composables mostly presentational and small enough for independent previews/tests.

Exact code change where possible:

```
admin/
  home/AdminHomeScreen.kt
  inventory/CategoryScreen.kt
  cart/OpenCartScreen.kt
  deals/SavedDealsScreen.kt
  media/MediaGalleryPopup.kt
  model/AdminUiState.kt
```

Test to confirm the fix: Track clean and incremental compile times before/after the split. Verify the project compiles reliably with standard 2?4 GiB Kotlin heap settings.

Medium Findings

1. Startup is network-gated and auth failure is swallowed

Severity: Medium
Location: app/src/main/java/com/ashokvatika/MainActivity.kt - onCreate(), ensureFirebaseSession()

Diagnosis: `setContent` is called only after `ensureFirebaseSession()` returns inside a coroutine. On slow or broken networks, first render is delayed. The anonymous sign-in failure is wrapped in `runCatching` and ignored, so the app can silently continue into a broken data state with no explicit UX fallback.

Impact: UX, perceived performance, supportability, offline resilience.

Suggested amendment: Render the UI immediately and expose auth/sync state separately. Report anonymous-auth failures in UI and avoid blocking initial composition on network I/O.

Exact code change where possible:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent { App() }
    lifecycleScope.launch {
        authState.value = runCatching { ensureFirebaseSession() }.isSuccess
    }
}
```

Test to confirm the fix: Launch the app in airplane mode. Expected after fix: UI appears immediately with a visible offline/auth error state instead of a delayed or silent start.

2. Image loading repeatedly decodes local URIs without a cache layer

Severity: Medium
Location: app/src/main/java/com/ashokvatika/ui/admin/AdminScreens.kt - rememberDecodedImage(), decodeImageBitmap()

Diagnosis: Many surfaces call `rememberDecodedImage()` directly, which decodes bitmaps from `ContentResolver` on demand but does not use an image-loading library or memory/disk cache. Repeated scrolling across OTC, cart, and media templates can trigger duplicate decodes and jank.

Impact: Performance, memory pressure, scroll smoothness, battery.

Suggested amendment: Use Coil or another Compose-aware image loader with caching, size negotiation, and lifecycle management.

Exact code change where possible:

```
// build.gradle.kts
implementation("io.coil-kt:coil-compose:2.7.0")

// UI
AsyncImage(
    model = ImageRequest.Builder(context)
        .data(item.imageUri)
        .crossfade(true)
        .size(Size.ORIGINAL)
        .build(),
    contentDescription = item.name,
    contentScale = ContentScale.Crop,
)
```

Test to confirm the fix: Profile scrolling in OTC and Open Cart with Android Studio profiler. Expected after fix: lower decode churn, fewer GC spikes, and smoother frame times.

3. Transaction code generation is collision-prone

Severity: Medium
Location: app/src/main/java/com/ashokvatika/ui/AppRoot.kt - generateTransactionCode()

Diagnosis: Transaction codes are generated as `TX-` plus the last 6 digits of `System.currentTimeMillis()`. That value repeats roughly every 16 minutes and can collide across devices instantly. For a business-facing identifier shown to users, that is too weak.

Impact: Auditability, support operations, reconciliation risk, UX confusion.

Suggested amendment: Use a collision-resistant transaction reference: full timestamp with date prefix, server-generated ID, or UUID-shortcode strategy.

Exact code change where possible:

```
private fun generateTransactionCode(): String {
    val ts = java.time.ZonedDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss"))
    val nonce = java.util.UUID.randomUUID().toString().take(8).uppercase()
    return "TX-$ts-$nonce"
}
```

Test to confirm the fix: Generate thousands of transaction codes across repeated sessions and devices. Expected after fix: no duplicates in sampled output.

4. Automated test coverage is effectively absent

Severity: Medium
Location: Repository structure - no app/src/test or app/src/androidTest suites present

Diagnosis: The project currently has no visible unit-test or instrumentation-test coverage for login, inventory persistence, cart totals, payment gating, or image/media flows. With UI logic concentrated in Compose and business logic embedded in composables, regressions are likely to ship unnoticed.

Impact: QA risk, release confidence, maintainability, delivery speed.

Suggested amendment: Add a minimum release gate: unit tests for pricing and transaction generation, repository tests for Firestore mapping, and instrumentation tests for login/cart/deal flows.

Exact code change where possible:

```
// Example unit test target areas
- generateTransactionCode uniqueness
- InventoryItem <-> Firestore mapping
- OpenCartRow.displayAmount()
- Payment close button enabled state
```

Test to confirm the fix: CI should run `testDebugUnitTest` and at least one smoke instrumentation suite before release builds are accepted.

Unverified Items

1. Firestore security rules are not in the repository and could not be verified

Location: Backend / Firebase project configuration

Diagnosis: Unverified. The client signs in anonymously and performs inventory and access-request writes directly from the device. If Firestore rules are permissive, any anonymous client could read or mutate business data.

How to verify: Inspect `firestore.rules` in source control or Firebase console. Confirm least-privilege rules for anonymous users, admin-only writes, and collection-level restrictions for `inventory_items` and `access_requests`.

2. Local bridge/provider integration appears incomplete

Location: app/src/main/AndroidManifest.xml vs data/LocalBridgeProvider.kt

Diagnosis: Unverified. A full ContentProvider/SQLite bridge exists in code, but no corresponding `LocalBridgeProvider` declaration is present in the manifest. If this bridge is expected for integrations or offline sync, it is currently non-functional. If not needed, it is dead code that increases maintenance surface.

How to verify: Decide whether the bridge is a product requirement. If yes, register and test it. If no, remove it and its contract/database classes.

Summary

The codebase is feature-rich but not production-ready yet. The most urgent blockers are hardcoded admin access, a non-verified payment closeout path, volatile in-memory sales state, and pricing fields that are calculated but not persisted. Build stability and image handling also need attention before release.